

SQL Commands

Standard SQL that works across PostgreSQL, MySQL, and SQLite unless noted. Keywords are shown uppercase by convention; SQL itself is case-insensitive for keywords.

Querying

Clause	What it does
<code>SELECT col1, col2</code>	pick columns (<code>*</code> for all)
<code>FROM table</code>	the source table
<code>WHERE cond</code>	filter rows before grouping
<code>ORDER BY col DESC</code>	sort results (default <code>ASC</code>)
<code>LIMIT 10 OFFSET 20</code>	paginate: 10 rows, skipping 20
<code>SELECT DISTINCT col</code>	unique values only
<code>AS alias</code>	rename a column or table in the output

A typical read query

```
SELECT name, email, created_at
FROM users
WHERE active = true
ORDER BY created_at DESC
LIMIT 25;
```

Filtering

Operator	Matches
<code>col = 5</code>	exact equality
<code>col <> 5</code>	not equal (also <code>≠</code>)
<code>col LIKE 'a%'</code>	starts with a (<code>%</code> = any, <code>_</code> = one char)
<code>col ILIKE 'a%'</code>	case-insensitive LIKE (Postgres)
<code>col IN (1, 2, 3)</code>	matches any value in the list
<code>col BETWEEN 1 AND 9</code>	inclusive range
<code>col IS NULL</code>	test for NULL (never use <code>= NULL</code>)
<code>a = 1 AND b = 2</code>	combine conditions (<code>OR</code> , <code>NOT</code> too)

Joins

Join	Returns
<code>INNER JOIN</code>	only rows matching in both tables
<code>LEFT JOIN</code>	all left rows + matches (NULLs if none)
<code>RIGHT JOIN</code>	all right rows + matches
<code>FULL OUTER JOIN</code>	all rows from both sides
<code>CROSS JOIN</code>	every combination (Cartesian product)

Join with an aggregate

```
SELECT u.name, COUNT(o.id) AS orders
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.name
ORDER BY orders DESC;
```

Aggregation

Function / clause	Purpose
<code>COUNT(*)</code>	number of rows
<code>SUM(col)</code>	total of a numeric column
<code>AVG(col)</code>	average
<code>MIN(col) / MAX(col)</code>	smallest / largest
<code>GROUP BY col</code>	collapse rows into groups
<code>HAVING COUNT(*) > 1</code>	filter groups (WHERE filters rows)

WHERE vs HAVING: WHERE filters individual rows *before* grouping; HAVING filters *after* grouping, so it can reference aggregates like `COUNT(*)` .

Modifying Data

Insert, update, delete

```
INSERT INTO users (name, email)
VALUES ('Ada', 'ada@x.io');

UPDATE users SET active = false
WHERE last_seen < '2025-01-01';

DELETE FROM users WHERE id = 42;
```

Upsert (Postgres / SQLite)

```
INSERT INTO users (email, name)
VALUES ('ada@x.io', 'Ada')
ON CONFLICT (email)
DO UPDATE SET name = EXCLUDED.name;
```

Always pair UPDATE / DELETE with a WHERE. Without one, you change *every* row. Run the matching `SELECT` first, and wrap risky changes in a `BEGIN; ... ROLLBACK;` transaction while you check.

Tables & Indexes

Create a table

```
CREATE TABLE orders (  
  id          SERIAL PRIMARY KEY,  
  user_id     INTEGER REFERENCES users(id),  
  total       NUMERIC(10,2) NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT now()  
);
```

Statement	Effect
ALTER TABLE t ADD COLUMN c TEXT	add a column
ALTER TABLE t DROP COLUMN c	remove a column
CREATE INDEX ON t (col)	speed up lookups on col
DROP TABLE t	delete a table and its data